Deliverable1

October 13, 2026

1 Deliverable 1

1.0.1 Task 1

[79]:
```
import locale
import sys

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd
import requests
from numpy import mean, median


def load_frost_key(path="../key_frost.py"):
    """Load API client ID from local file."""
    sys.path.append("..")
    with open(path, "r", encoding="utf-8") as file:
        return file.read().strip()


def get_frost_data(client_id, year):
    """Connecting to frost server and get the mean temperatur for 2025."""
    endpoint = "https://frost.met.no/observations/v0.jsonld"
    parameters = {
        "sources": "SN17850",
        "elements": "mean(air_temperature P1D)",
        "referencetime": f"{year}-01-01/{year}-12-31",
    }

    response = requests.get(
        endpoint,
        params=parameters,
        auth=(client_id, "")
    )
    response.raise_for_status()
    return response.json()["data"]
```

```python
def get_the_rigth_format(data):
    """Collect the dates and temperatures from Frost API response and get the␣
 ↪dates in right format."""
    dates = []
    temps = []

    for entry in data:
        date = entry["referenceTime"][:10]  # YYYY-MM-DD
        temp = entry["observations"][0]["value"]
        dates.append(date)
        temps.append(temp)

    df = pd.DataFrame({
        "Dato": pd.to_datetime(dates),
        "Temperatur": temps
    })
    df["Dag"] = df["Dato"].dt.dayofyear
    return df


def summary(df):
    """Creating a summary for min, max, mean and median temperatur."""
    min_temp = df["Temperatur"].min()
    max_temp = df["Temperatur"].max()
    mean_temp = mean(df["Temperatur"])
    median_temp = median(df["Temperatur"])

    min_date = df.loc[df["Temperatur"].idxmin(), "Dato"].date()

    max_date = df.loc[df["Temperatur"].idxmax(), "Dato"].date()
    print("Summary for 2025:\n")
    print(
        f"Min temperature: {min_temp} °C, measured {min_date}\n"
        f"Max temperature: {max_temp} °C, measured {max_date}"
    )
    print(f"Mean: {mean_temp:.1f} °C")
    print(f"Median: {median_temp:.1f} °C\n")



def plot_temperature(df):
    """Plot temperature series with Norwegian month labels."""
    plt.figure(figsize=(12, 5))
    plt.plot(df["Dato"], df["Temperatur"], color="steelblue", linewidth=1.8,␣
 ↪markersize=3)
```

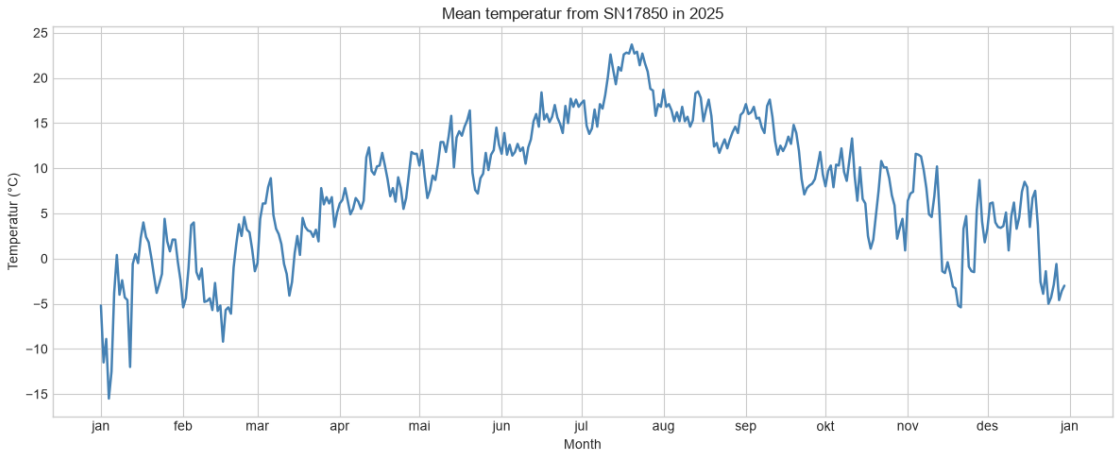
```
    locale.setlocale(locale.LC_TIME, "nb_NO.UTF-8")

    ax = plt.gca()
    ax.xaxis.set_major_locator(mdates.MonthLocator())
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))

    plt.xticks(rotation=0)
    plt.xlabel("Month")
    plt.ylabel("Temperatur (°C)")
    plt.title("Mean temperatur from SN17850 in 2025")
    plt.tight_layout()
    plt.show()


def main():
    """The main program that runs task 1 and collect the right data
    and specifies which data each function shall use."""
    client_id = load_frost_key()
    data = get_frost_data(client_id, 2025)
    df = get_the_rigth_format(data)
    plot_temperature(df)
    summary(df)

if __name__ == "__main__":
    main()
```




```
Summary for 2025:

Min temperature: -15.5 °C, measured 2025-01-04
Max temperature: 23.7 °C, measured 2025-07-20
Mean: 7.9 °C
```

Median: 8.4 °C

1.0.2 Results

We think the graph looks good, and is understandable to read.

1.0.3 Task 2

[80]:
```
def get_frost_data_rain(client_id):
    """Connctin to frost server and get the precipitation for 2025."""
    endpoint = "https://frost.met.no/observations/v0.jsonld"
    parameters = {
        "sources": "SN17850",
        "elements": "sum(precipitation_amount P1D)",
        "referencetime": "2025-01-01/2025-12-31",
    }

    response = requests.get(
        endpoint,
        params=parameters,
        auth=(client_id, "")
    )
    response.raise_for_status()
    return response.json()["data"]

def get_the_rigth_format_rain(data):
    """Collect rain from Frost API response."""
    rain = []

    for entry in data:
        rains = entry["observations"][0]["value"]
        rain.append(rains)

    return rain

def plot_temperature_rain(df, rain):
    """Plot rain series with Norwegian month labels."""

    fig, ax1 = plt.subplots(figsize=(12, 5))

    # plott in the temperature
    ax1.plot(df["Dato"], df["Temperatur"], color="blue", label="Temperature␣
 ↪(°C)")
    ax1.set_xlabel('Month')
    ax1.set_ylabel('Temperature (°C)', color='blue')
    ax1.tick_params(axis='y', labelcolor='blue')
```

```
    # plott in the rain
    ax2 = ax1.twinx()

    ax2.set_ylim(0, 45)
    ax1.set_ylim(-20, 25)

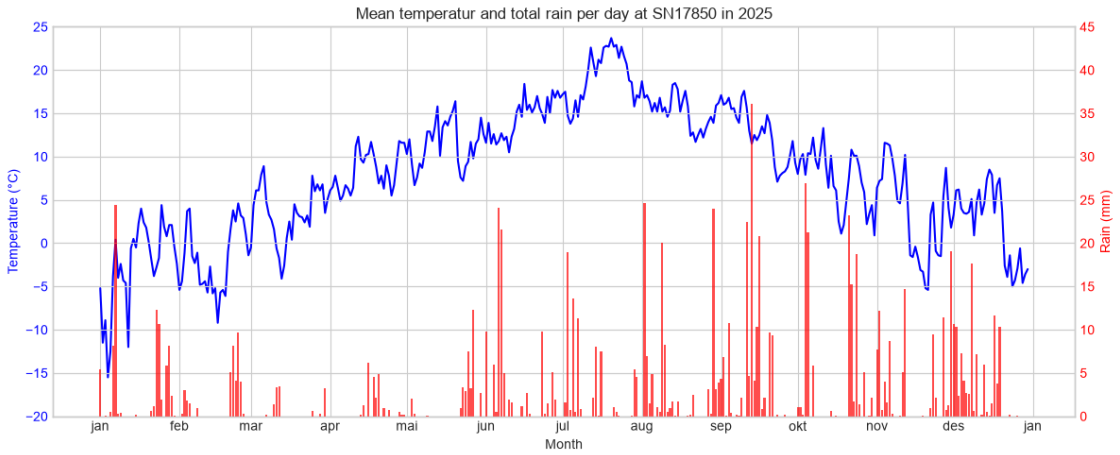
    ax2.bar(df["Dato"], rain, color="red", alpha=0.7, label="Rain (mm)")
    ax2.set_ylabel("Rain (mm)", color="red")
    ax2.tick_params(axis="y", labelcolor="red")


    locale.setlocale(locale.LC_TIME, "nb_NO.UTF-8")
    ax = plt.gca()
    ax.xaxis.set_major_locator(mdates.MonthLocator())
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))


    plt.xticks(rotation=0)
    plt.title("Mean temperatur and total rain per day at SN17850 in 2025")
    fig.tight_layout()
    plt.show()

def main():
    client_id = load_frost_key()
    data = get_frost_data(client_id, 2025)
    rdata = get_frost_data_rain(client_id)
    df = get_the_rigth_format(data)
    rain = get_the_rigth_format_rain(rdata)
    plot_temperature_rain(df, rain)

if __name__ == "__main__":
    main()
```



1.0.4 Results

We chose to use a graph for temperature and bars for rain, we managed to line ut the grid in th

1.0.5 Task 3

[81]:
```
def summary2(df):
    """Creating a summary for min, max, mean and median temperatur."""
    min_temp = df["Temperatur"].min()
    max_temp = df["Temperatur"].max()
    mean_temp = mean(df["Temperatur"])
    median_temp = median(df["Temperatur"])

    min_date = df.loc[df["Temperatur"].idxmin(), "Dato"].date()

    max_date = df.loc[df["Temperatur"].idxmax(), "Dato"].date()
    print("Summary for 1925:\n")
    print(
        f"Lowest temperatur: {min_temp} °C, measured {min_date}\n"
        f"Max temperatur: {max_temp} °C, measured {max_date}"
    )
    print(f"Mean: {mean_temp:.1f} °C")
    print(f"Median: {median_temp:.1f} °C\n")


def plot_2_temperature(df_year1, df_year2):
    """Plot temperature series with Norwegian month labels."""
    plt.style.use("seaborn-v0_8-whitegrid")
    plt.figure(figsize=(12, 5))
    plt.plot(df_year1["Dag"], df_year1["Temperatur"], label="1925", linewidth=1.
 ↪8, markersize=3, color="b")
    plt.plot(df_year2["Dag"], df_year2["Temperatur"], label="2025", linewidth=1.
 ↪8, markersize=3, color="r")

    month_names = ("jan", "feb", "mar", "apr", "may", "jun", "jul", "aug",␣
 ↪"sep", "okt", "nov", "des")
    month_starts = (1, 32, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335)
    plt.xticks(month_starts, month_names)
    plt.xlim(1, 365)
    plt.xlabel("Month")
    plt.ylabel("Temperature (°C)")
    plt.title("Compare 1925 mean temperature with 2025 from sources SN17850")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

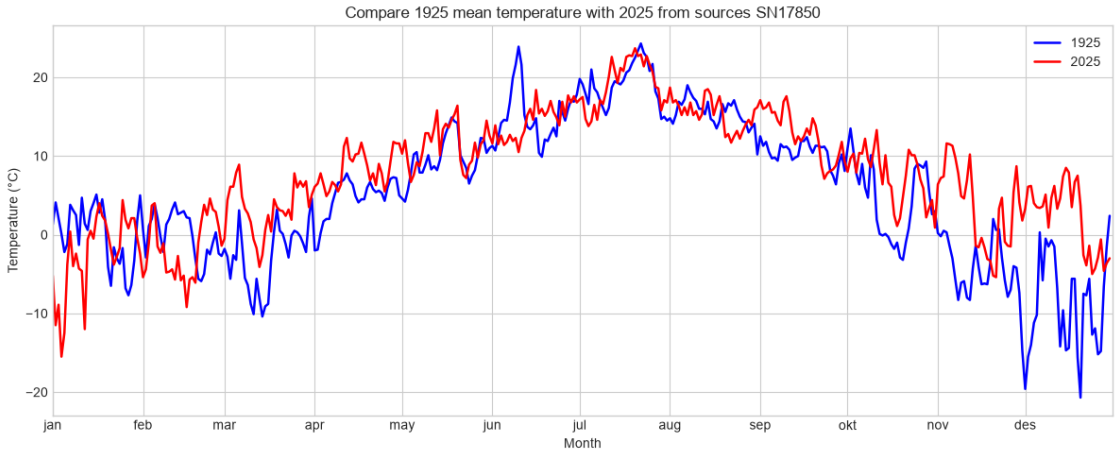
```python
def main():
    """The main program that runs task 3 and collect the right data
    and specifies which data each function shall use."""
    client_id = load_frost_key()

    data_1925 = get_frost_data(client_id, 1925)
    data_2025 = get_frost_data(client_id, 2025)

    df_1925 = get_the_rigth_format(data_1925)
    df_2025 = get_the_rigth_format(data_2025)

    plot_2_temperature(df_1925, df_2025)
    summary2(df_1925)
    summary(df_2025)


if __name__ == "__main__":
    main()
```




```
Summary for 1925:

Lowest temperatur: -20.7 °C, measured 1925-12-20
Max temperatur: 24.3 °C, measured 1925-07-22
Mean: 5.3 °C
Median: 5.3 °C

Summary for 2025:

Min temperature: -15.5 °C, measured 2025-01-04
Max temperature: 23.7 °C, measured 2025-07-20
```

```
Mean: 7.9 °C
Median: 8.4 °C
```

1.0.6 Results

We chose to use two very different colors for the graphs, as you can se the temperature is not

1.0.7 Task 4

[82]:
```python
import yaml


def get_frost_maxdata(client_id, station_id, year):
    """Connect to Frost API and define that we want the max daily temperature␣
 ↪for 2025."""
    endpoint = "https://frost.met.no/observations/v0.jsonld"
    parameters = {
        "sources": station_id,
        "elements": "max(air_temperature P1D)",
        "referencetime": f"{year}-01-01/{year}-12-31",
    }

    response = requests.get(endpoint, params=parameters, auth=(client_id, ""))
    response.raise_for_status()
    return response.json()["data"]


def heatwave (data):
    """Counting how many heatwaves there are in the current period.
    A heatwave is a periode with 5 or more days above 27°C."""

    heatwaves = []
    current_streak = []

    for entry in sorted(data, key=lambda item: item["referenceTime"]):
        day = entry["referenceTime"][:10]
        temp = entry["observations"][0]["value"]

        if temp >= 27:
            current_streak.append(day)
        else:
            if len(current_streak) >= 5:
                heatwaves.append(f"{current_streak[0]} - {current_streak[-1]}")
            current_streak = []

    if len(current_streak) >= 5:
        heatwaves.append(f"{current_streak[0]} - {current_streak[-1]}")
```

```
    return heatwaves


def yml_file(station_id, year, heatwaves):
    """Writes a yml file with the right format"""
    data = {
        "dates": heatwaves,
        "station": station_id,
        "year": year,
        "heatwave_count": len(heatwaves),
    }

    with open("heatwave_summary.yml", "w", encoding="utf-8") as output_file:
        yaml.safe_dump(data, output_file, sort_keys=False)


def main():
    """The main program that runs task 4 and collect the right data
    and specifies which data each function shall use."""
    client_id = load_frost_key()
    station_id = "SN17850"
    year = 2025

    max_data = get_frost_maxdata(client_id, station_id, year)
    heatwaves = heatwave(max_data)
    yml_file(station_id, year, heatwaves)


# We tried to locate the name of the station, so we printed all the elements in␣
 ↪the data, but we didnt find a name
# We found the name on the web page, named "Ås"
#print(max_data[0])



if __name__ == "__main__":
    main()
```

1.0.8 Results

The yaml file we made is in the right format as in the example, but we couldnt manage to get th

1.0.9 Task 5

[83]:
```
def plot_temperature_t5(df):
    """Plot temperature series with Norwegian month labels."""
    df["7_day_av"] = df["Temperatur"].rolling(window=7).mean()
    plt.figure(figsize=(12, 5))
    plt.plot(df["Dato"], df["7_day_av"], color="steelblue", linewidth=1.8,␣
 ↪label="7 day avg")
    plt.plot(df["Dato"], df["Temperatur"], color="red", marker="o",␣
 ↪linewidth=0, markersize =2, label="mean temp that day")

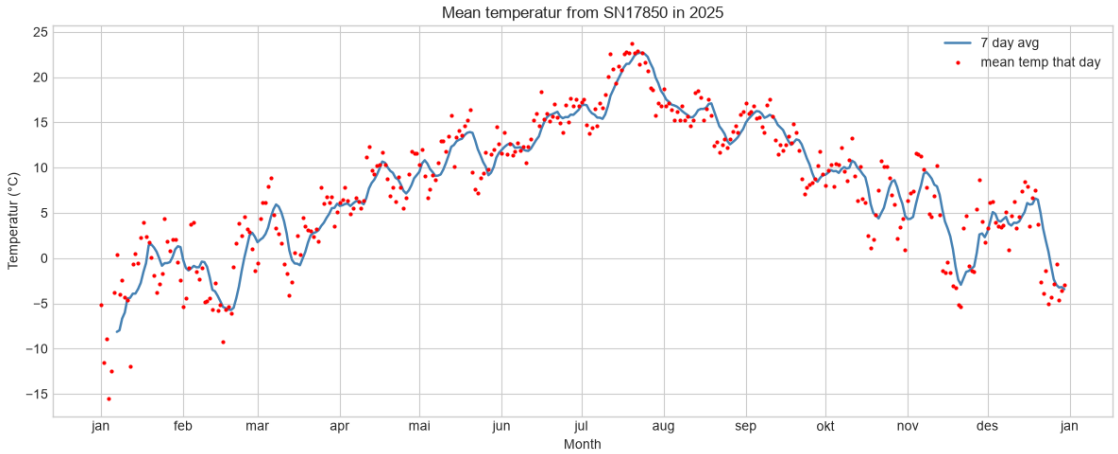
    locale.setlocale(locale.LC_TIME, "nb_NO.UTF-8")

    ax = plt.gca()
    ax.xaxis.set_major_locator(mdates.MonthLocator())
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))

    plt.xticks(rotation=0)
    plt.xlabel("Month")
    plt.ylabel("Temperatur (°C)")
    plt.title("Mean temperatur from SN17850 in 2025")
    plt.legend()
    plt.tight_layout()
    plt.show()


def main():
    """The main program that runs task 5 and collect the right data
    and specifies which data each function shall use."""
    client_id = load_frost_key()
    data = get_frost_data(client_id, 2025)
    df = get_the_rigth_format(data)
    plot_temperature_t5(df)


if __name__ == "__main__":
    main()
```



1.0.10 Results

We managed to make the right graph, it was easy to reuse the data from task 1.

When we follow Pep-8 it was easy to reuse the functions in every task.

1.0.11 sources

Norwegian Meteorological Institute. (2026, 13/10) Frost.met.no. https://frost.met.no/observatio

Github repository: https://github.com/GuavaIste/Deliverable-1

1.1 Ai-chatlogs

Task 1 was first written by hand with inspiration from the lectures and the task was without a
We used ai to help us with Pandas and format the data.
I wrote a summary and used ai to help me make it better.
We used ai to help make the plots, so they looked good.
Ai knows a lot of very smart and uniqe methods to make coding easier in a lot of instances. fo